PROYECTO FINAL – LA AVENTURA DEL SABIO

PROGRAMACION ORIENTADO A OBJETOS

INTEGRANTES:

BENITES CORAZON, MAX ANDERSON

CARLOS CAJO, JOSE MANUEL

VELASQUEZ RIMAC, LUIS

DOCENTE: CABRERA DIAZ, JAVIER ELMER

SECCION: 35146

Diciembre 2025

Historial de Versiones

VERSIÓN	PARTES QUE CAMBIAN	DESCRIPCIÓN DEL CAMBIO	FECHA DE CAMBIO	MODIFICADO POR	APROBADO POR
1.0			04/11/2025	Max BenitesC	
2.0			**25/11/2025**	Jose Carlos	
3.0			02/12/2025	Luis Velasquez	

Índice

HISTORIAL DE VERSIONES 2
ÍNDICE 3
1 INTRODUCCIÓN 4
2 ALCANCE 5
3 CAPTURA DEL CODIGO ..... ¡ERROR! MARCADOR NO DEFINIDO.
4 EJECUCION DEL CODIGO 18
5 CONCLUSIONES 21
6 RECOMENDACIONES 21

1. INTRODUCCIÓN

El presente proyecto individual consiste en el diseño y desarrollo de un videojuego educativo por turnos, titulado “La Aventura del Sabio”, desarrollado en Java bajo los principios de la Programación Orientada a Objetos (POO). Este juego combina elementos de rol y trivia, permitiendo que los jugadores refuercen sus conocimientos generales mediante preguntas clasificadas por nivel de dificultad.

Cada jugador representa a un personaje del tipo “Sabio”, vinculado a un elemento (Fuego, Agua o Tierra), y debe enfrentarse a rondas de preguntas para ganar puntos de experiencia (XP). A medida que acumula XP, el sabio sube de nivel y mejora su rendimiento, hasta completar la misión final.

El sistema está diseñado como una aplicación de consola, en modo por turnos, para 1 a 3 jugadores en una misma sesión. Este proyecto permite demostrar el uso de conceptos clave de la POO como clases abstractas, interfaces, herencia, polimorfismo, encapsulamiento, así como el uso de colecciones y validaciones de flujo de datos.

2. ALCANCE

Este proyecto implementa un juego por turnos donde los jugadores compiten respondiendo preguntas clasificadas por dificultad, acumulando experiencia y progresando en niveles. Se centra en el uso correcto de los pilares de la programación orientada a objetos, demostrando su aplicación en un entorno funcional y educativo.

2.1 Delimitaciones del sistema:

- El sistema se ejecuta como una aplicación de consola (sin interfaz gráfica).
- Admite de 1 a 3 jugadores por sesión.
- Cada jugador tiene nombre, clase, puntos de salud (PS), experiencia (XP), aciertos y errores.
- Las preguntas están clasificadas y almacenadas en arreglos o colecciones (no se cargan desde archivos externos).
- La lógica de juego se basa en turnos rotativos.
- No se implementa persistencia de datos (no hay base de datos ni archivos guardados).
- Se controlan errores comunes de entrada por teclado.
- Se utiliza herencia múltiple simulada mediante una interfaz Atacante y una clase abstracta Personaje

2.2 Tema 1: Módulo de gestión de jugadores

- Solicita al usuario la cantidad de jugadores.
- Permite registrar nombre y tipo de personaje (Sabio de Fuego, Agua o Tierra).
- Instancia objetos de clase SabioFuego, SabioAgua o SabioTierra usando new.
- Valida la selección del jugador.
- Se aplican principios de encapsulamiento para proteger atributos como XP o PS.

2.3 Tema 2: Módulo de preguntas y validación

- Las preguntas están divididas en niveles de dificultad: fácil, media y difícil.
- Se presentan preguntas distintas a cada jugador, sin repetir.
- Cada pregunta tiene opciones múltiples (A, B, C).
- El sistema evalúa la respuesta del jugador, notifica si fue correcta o incorrecta y muestra la respuesta correcta cuando se falla.
- Se suma XP según el nivel de la pregunta acertada.

2.4 Tema 3: Módulo de progreso, experiencia y resultados

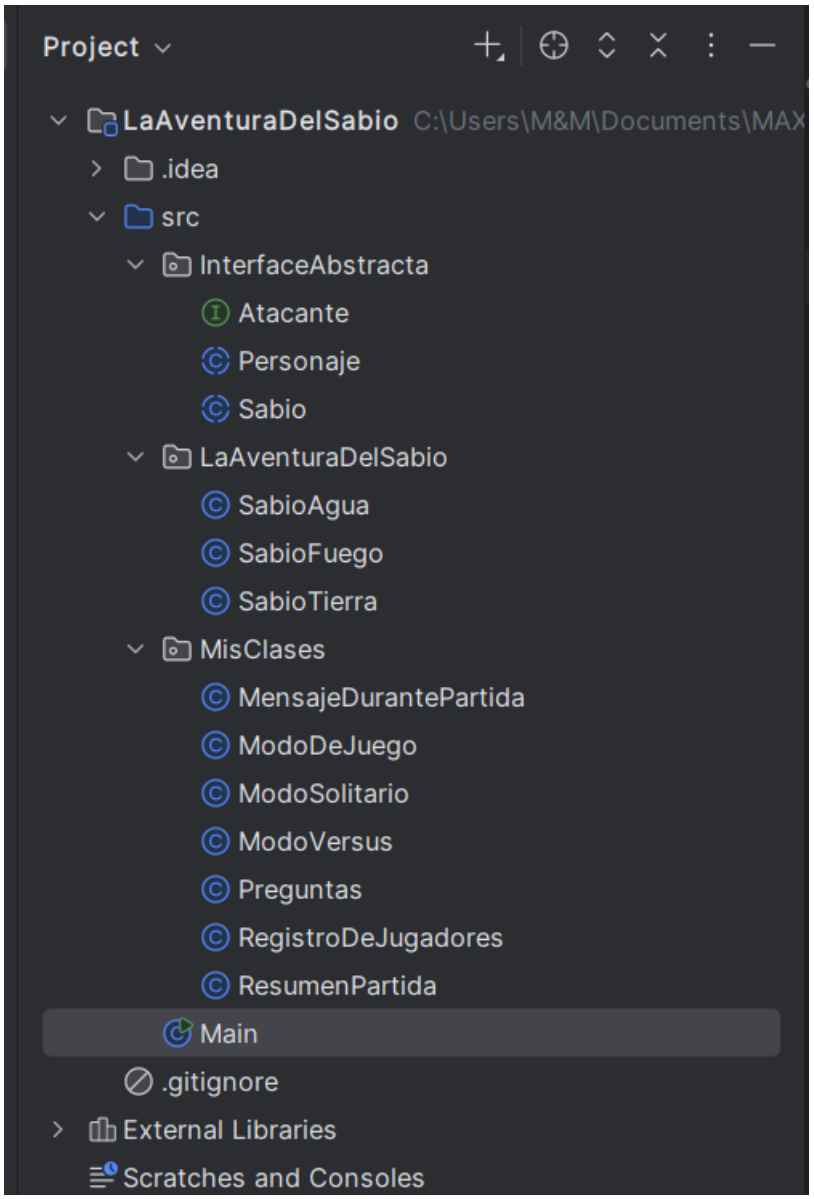
- Al responder correctamente, el jugador gana puntos de experiencia (XP).
- El nivel del jugador se calcula en función del XP acumulado.
- Cuando un jugador sube de nivel, se muestra un mensaje especial.
- Al final del juego se muestra un resumen final de cada jugador: XP, nivel alcanzado, aciertos y errores.
- Se identifican los jugadores que completaron la misión (nivel máximo).
- Se anuncia un ganador o se declara empate según los resultados.

2.5 Tema 4: Módulo de interfaz y narrativa del juego

- La narrativa del juego varía según la clase del sabio: Fuego, Agua o Tierra.
- Se utilizan mensajes temáticos en consola para dar personalidad a cada sabio.
- Se muestran mensajes especiales al subir de nivel o completar la misión.
- Se indica el turno actual y se guía al jugador en todo momento.

3. CAPTURAS DEL CODIGO:

Interface, Abstracta y clases concretas:



CLASE MAIN:

```java
import MisClases.*;
import InterfaceAbstracta.*;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        ModoDeJuego mododejuego=new ModoDeJuego();
        do{
            mododejuego.eleccionModoJuego();
            if (mododejuego.getModoJuego() == 1) {
                List<Sabio> jugadores = RegistroDeJugadores.registrarJugador( numJugadores: 1);
                RegistroDeJugadores.mostrarInfoJugador(jugadores);
                MensajeDurantePartida.mostrarInicioPartida();
                ModoSolitario modosolitario = new ModoSolitario();
                modosolitario.jugarModoSolitario(jugadores.get(0));
            } else if (mododejuego.getModoJuego() == 2){
                int numJugadores = mododejuego.getNumJugadores();
                List<Sabio> jugadoresVersus = RegistroDeJugadores.registrarJugador(numJugadores);
                RegistroDeJugadores.mostrarInfoJugador(jugadoresVersus);
                MensajeDurantePartida.mostrarInicioPartida();
                ModoVersus modoversus = new ModoVersus();
                modoversus.jugarModoVersus(jugadoresVersus, numJugadores);
            }
        }while(mododejuego.getModoJuego()!=9);
    }
}
```

CLASE INTERFACE Atacante:

```java
package InterfaceAbstracta;

public interface Atacante {  no usages  5 implementations
    void atacar(Personaje objetivo, int danoBase);  no usages  4 implementations
}
```

CLASE ABSTRACTA Personaje:

```
Personaje.java ×   Sabio.java   SabioAgua.java   SabioFuego.java   SabioTierra.java   MensajeDura

1   package InterfaceAbstracta;
2
3   public abstract class Personaje implements Atacante {  10 usages  4 inheritors
4       private String nombre;  2 usages
5       private String rol;  2 usages
6       private int ps;  6 usages
7       private int xp;  3 usages
8
9       protected Personaje(String nombre, String rol, int psInicial) {  1 usage
10          this.nombre = nombre;
11          this.rol = rol;
12          this.ps = psInicial;
13          this.xp = 0;
14      }
15
16      @Override  2 usages  3 implementations
17      public abstract void atacar(Personaje objetivo, int danoBase);
18
19      public void sumarXp(int valor) {  2 usages
20          this.xp += valor;
21      }
22
23      public void recibirDano(int valor) {  7 usages
24          this.ps -= valor;
25          if (this.ps < 0) {
26              this.ps = 0;
27              }
28          }
29
30          public boolean estaVivo() {  10 usages
31              return this.ps > 0;
32          }
33
34          public String getNombre() {  16 usages
35              return nombre;
36          }
37
38          public String getRol() {  4 usages
39              return rol;
40          }
41
42          public int getPs() {  5 usages
43              return ps;
44          }
45
46          public int getXp() {  5 usages
47              return xp;
48          }
49      }
```

CLASE ABSTRACTA Sabio:

```java
package InterfaceAbstracta;

public abstract class Sabio extends Personaje {  28 usages  3 inheritors

    private int nivelSabiduria;  2 usages

    protected Sabio(String nombre, String rol, int psInicial, int sabiduria) {
        super(nombre, rol, psInicial);
        this.nivelSabiduria = sabiduria;
    }

    public int getNivelSabiduria() {  no usages
        return nivelSabiduria;
    }
}
```

CLASES CONCRETAS

SabioAgua:

```java
package LaAventuraDelSabio;

import InterfaceAbstracta.*;

public class SabioAgua extends Sabio {  1 usage

    public SabioAgua(String nombre) {  1 usage
        super(nombre,  rol: "Agua",  psInicial: 70,  sabiduria: 7);
    }

    @Override  2 usages
    public void atacar(Personaje objetivo, int danoBase) {
        int danoAjustado = danoBase - 2;
        objetivo.recibirDano(danoAjustado);
        System.out.println("💧 ¡" + getNombre() + " (Agua) usó Chorro de" +
                " Agua! Causó " + danoAjustado + " de daño a " + objetivo.getNombre() + ".");
    }
}
```

SabioFuego:

```java
package LaAventuraDelSabio;

import InterfaceAbstracta.*;

public class SabioFuego extends Sabio {  2 usages

    public SabioFuego(String nombre) {  2 usages
        super(nombre, rol: "Fuego", psInicial: 100, sabiduria: 5);
    }

    @Override  2 usages
    public void atacar(Personaje objetivo, int danoBase) {
        int danoAjustado = danoBase + 5;
        objetivo.recibirDano(danoAjustado);
        System.out.println("🔥 ¡" + getNombre() + " (Fuego) usó Erupción! " +
                "Causó " + danoAjustado + " de daño a " + objetivo.getNombre() + ".");
    }
}
```

SabioTierra:

```java
package LaAventuraDelSabio;

import InterfaceAbstracta.*;

public class SabioTierra extends Sabio {  1 usage

    public SabioTierra(String nombre) {  1 usage
        super(nombre, rol: "Tierra", psInicial: 90, sabiduria: 6);
    }

    @Override  2 usages
    public void atacar(Personaje objetivo, int danoBase) {
        int danoAjustado = danoBase + 2;
        objetivo.recibirDano(danoAjustado);
        System.out.println("🪨 ¡" + getNombre() + " (Tierra) usó Roca " +
                "Sólida! Causó " + danoAjustado + " de daño a " + objetivo.getNombre() + ".");
    }
}
```

MISCLASES:

MensajeDurantePartida:

```
MensajeDurantePartida.java   ModoDeJuego.java   ModoSolitario.java   ModoVersus.java   Preguntas.java   RegistroDeJugadores.java
```

```java
package MisClases;
public class MensajeDurantePartida {  8 usages
    public static void mostrarMenuPrincipal() {  1 usage
        System.out.println("╔═══════════════════════════════════════════╗");
        System.out.println("│          LA AVENTURA DEL SABIO            │");
        System.out.println("╟───────────────────────────────────────────╢");
        System.out.println("│ 1. Solitario (contra la máquina)          │");
        System.out.println("│ 2. Versus (2 o 3 jugadores)               │");
        System.out.println("│ 9. Salir del Juego                        │");
        System.out.println("╚═══════════════════════════════════════════╝");
    }

    public static void mostrarRegistroJugador(int i) {  1 usage
        System.out.println("\n╔═══════════════════════════╗");
        System.out.println("│    REGISTRO DE JUGADOR " + (i + 1) + "    │");
        System.out.println("╚═══════════════════════════╝");
    }

    public static void mostrarElementosSabio(String nombre) {  1 usage
        System.out.println("Hola SABIO " + nombre.toUpperCase() + " ¿A que elemento quieres pertenecer?:");
        System.out.println("┌─────┬──────────┐");
        System.out.println("│ [1] │ FUEGO    │");
        System.out.println("│ [2] │ AGUA     │");
        System.out.println("│ [3] │ TIERRA   │");
        System.out.println("└─────┴──────────┘");
    }

    public static void mostrarInicioPartida() {  2 usages
        System.out.println("\n═══════════════════════════════════════════");
        System.out.println("    LA AVENTURA COMIENZA... ¡PREPÁRATE! ");
        System.out.println("═══════════════════════════════════════════");
        System.out.println();
    }

    public static void mostrarFinalPartida() {  1 usage
        System.out.println("Partida finalizada! Gracias por jugar.");
    }

    public static void mostrarJugarNuevamente() {  2 usages
        System.out.println("\n¿Deseas volver a jugar?");
        System.out.println("[1] Sí");
        System.out.println("[2] Volver al menú principal");
    }
}
```

ModoDeJuego:

ModoDeJuego.java × ModoSolitario.java ModoVersus.java Preguntas.java RegistroDeJugadores.java ResumenP

```java
package MisClases;

import java.util.Scanner;

public class ModoDeJuego {  no usages
    private int modoJuego;  5 usages
    private int numJugadores;  7 usages

    public void eleccionModoJuego() {  no usages
        Scanner teclado = new Scanner(System.in);
        do{
            MensajeDurantePartida.mostrarMenuPrincipal();
            System.out.print("Elige una opción: ");
            modoJuego = teclado.nextInt();
            switch (modoJuego) {
                case 1:
                    System.out.println("---------Modo Solitario seleccionado---------");
                    numJugadores = 1;
                    break;
                case 2:
                    System.out.println("---------Modo Versus seleccionado---------");
                    do{
                        System.out.print("Ingrese cantidad de jugadores (2 o 3): ");
                        numJugadores = teclado.nextInt();
                        if(numJugadores != 2 && numJugadores != 3){
                            System.out.println("Cantidad de jugadores inválido! Elegir 2 o 3.");
                        }
                    }while(numJugadores != 2 && numJugadores != 3);
                    break;
                case 9:
                    MensajeDurantePartida.mostrarFinalPartida();
                    return;
                default:
                    System.out.println("Opción inválida. Intenta nuevamente.");
            }
        } while (modoJuego != 1 && modoJuego != 2);
    }

    public int getModoJuego() {  no usages
        return modoJuego;
    }

    public int getNumJugadores() {  no usages
        return numJugadores;
    }
}
```

ModoSolitario:

```java
package MisClases;
import InterfaceAbstracta.Sabio;
import java.util.Scanner;

public class ModoSolitario {  2 usages
    Preguntas generadorDePreguntas = new Preguntas();  3 usages
    private int respuestasCorrectas = 0;  2 usages
    private int respuestasIncorrectas = 0;  2 usages

    public void jugarModoSolitario(Sabio jugador){  2 usages
        Scanner teclado=new Scanner(System.in);
        char opcSeleccionada;
        for(int turno=0;turno<10;turno++){
            if (!jugador.estaVivo()) {
                System.out.println("¡" + jugador.getNombre() + " se ha quedado sin PS! Fin de la partida.");
                break;
            }
            do{
                generadorDePreguntas.mostrarPreguntaCompleta(turno, jugador, turno);
                System.out.print(" Elige tu respuesta: ");
                opcSeleccionada=teclado.next().toLowerCase().charAt(0);
                if (opcSeleccionada != 'a' && opcSeleccionada != 'b' && opcSeleccionada != 'c' && opcSeleccionada != 'd') {
                    System.out.println("\nOPCIÓN INVÁLIDA. Debes ingresar a, b, c o d.");
                }
            }while (opcSeleccionada != 'a' && opcSeleccionada != 'b' && opcSeleccionada != 'c' && opcSeleccionada != 'd');
            validarRespuesta(jugador,opcSeleccionada,turno);
        }
        ResumenPartida.mostrarResumenSolitario(jugador, respuestasCorrectas, respuestasIncorrectas);
        int decision;
        do {
            MensajeDurantePartida.mostrarJugarNuevamente();
            System.out.print("Elige una opción: ");
            decision = teclado.nextInt();
            if (decision == 1) {
                jugador.recibirDano( valor: -50);
                jugarModoSolitario(jugador);
            }
        } while (decision != 1 && decision != 2);
    }

    public void validarRespuesta(Sabio jugador, char respuestaUsuario, int turno) {  1 usage
        if (respuestaUsuario == generadorDePreguntas.getRespuesta(turno).charAt(0)) {
            System.out.println("¡Correcto! Ganas 10 XP.");
            jugador.sumarXp( valor: 10);
            respuestasCorrectas++;
        } else {
            System.out.println("Incorrecto. Pierdes 5 PS.");
            System.out.println("La respuesta correcta era: " + generadorDePreguntas.getRespuesta(turno));
            jugador.recibirDano( valor: 5);
            respuestasIncorrectas++;
        }
    }
}
```

ModoVersus:

```java
package MisClases;
import java.util.List;
import java.util.Scanner;
import InterfaceAbstracta.Personaje;
import InterfaceAbstracta.Sabio;

public class ModoVersus {  2 usages
    Preguntas generadorDePreguntas = new Preguntas();  3 usages
```

```java
    public void jugarModoVersus(List<Sabio> jugadores, int cantidadJugadores){  2 usages
        int[] respuestasCorrectas = new int[cantidadJugadores];
        int[] respuestasIncorrectas = new int[cantidadJugadores];
        Scanner teclado=new Scanner(System.in);
        char opcSeleccionada;
        for(int turno=0;turno<10;turno++){
            long jugadoresVivos = jugadores.stream().filter(Sabio::estaVivo).count();
            if (jugadoresVivos <= 1) {
                System.out.println("La partida ha terminado anticipadamente, solo queda un Sabio.");
                break;
            }
            for(int i =0;i<cantidadJugadores;i++){
                Sabio atacante = jugadores.get(i);
                if (!atacante.estaVivo()) {
                    System.out.println("\nEl Sabio " + atacante.getNombre() + " está K.O. y no puede participar.");
                    continue;
                }
                int numPregunta=turno*cantidadJugadores+i;
                do{
                    generadorDePreguntas.mostrarPreguntaCompleta(numPregunta, atacante, turno);
                    System.out.print(" Elige tu respuesta: ");
                    opcSeleccionada=teclado.next().toLowerCase().charAt(0);
                    if (opcSeleccionada != 'a' && opcSeleccionada != 'b' && opcSeleccionada != 'c' && opcSeleccionada != 'd') {
                        System.out.println("\nOPCIÓN INVÁLIDA. Debes ingresar a, b, c o d.");
                    }
                }while (opcSeleccionada != 'a' && opcSeleccionada != 'b' && opcSeleccionada != 'c' &&
                        opcSeleccionada != 'd');
                validarRespuesta(jugadores, atacante, i, opcSeleccionada, numPregunta, respuestasCorrectas, respuestasIncorrectas,
                        cantidadJugadores);
            }
        }
        ResumenPartida.mostrarResumenVersus(jugadores, respuestasCorrectas, respuestasIncorrectas);
        int decision;
        do {
            MensajeDurantePartida.mostrarJugarNuevamente();
            System.out.print("Elige una opción: ");
            decision = teclado.nextInt();
            if (decision == 1) {
                for(Sabio sabio : jugadores) {
                    sabio.recibirDano( valor: -1000);
                }
                jugarModoVersus(jugadores,cantidadJugadores);
            }
        } while (decision != 1 && decision != 2);
    }

    public void validarRespuesta(List<Sabio> jugadores, Sabio atacante, int atacanteIndex, char respuestaUsuario, int turno,  1 usage
                                 int[] respuestasCorrectas, int[] respuestasIncorrectas, int cantidadJugadores) {
        Scanner teclado = new Scanner(System.in);

        if (respuestaUsuario == generadorDePreguntas.getRespuesta(turno).charAt(0)) {
            System.out.println("¡Correcto! Ganas 10 XP.");
            atacante.sumarXp( valor: 10);
            respuestasCorrectas[atacanteIndex]++;
            int DANO_BASE = 10;
            if (cantidadJugadores == 2) {
                Sabio victima = (atacanteIndex == 0) ? jugadores.get(1) : jugadores.get(0);
                if (victima.estaVivo()) {
                    atacante.atacar(victima, DANO_BASE);
                } else {
                    System.out.println("El único rival restante está K.O.");
                }
            } else if (cantidadJugadores == 3) {
                int rivalElegidoIndex;
                Personaje rivalElegido = null;
                do {
                    System.out.println("\n¿A quién deseas atacar?");
                    for (int i = 0; i < cantidadJugadores; i++) {
                        if (i != atacanteIndex && jugadores.get(i).estaVivo()) {
                            System.out.println("[" + (i + 1) + "] " + jugadores.get(i).getNombre() + " (PS: " +
                                    jugadores.get(i).getPs() + ")");
                        }
                    }
```

```java
                System.out.print("Elige el número del jugador: ");
                rivalElegidoIndex = teclado.nextInt() - 1;
                if (rivalElegidoIndex >= 0 && rivalElegidoIndex < cantidadJugadores) {
                    rivalElegido = jugadores.get(rivalElegidoIndex);
                }
                if (rivalElegidoIndex == atacanteIndex || rivalElegido == null || !rivalElegido.estaVivo()) {
                    System.out.println("Opción inválida. Debes elegir un rival vivo y válido.");
                }
            } while (rivalElegidoIndex == atacanteIndex || rivalElegido == null || !rivalElegido.estaVivo());
            atacante.atacar(rivalElegido, DANO_BASE);
        }
    } else {
        System.out.println("Incorrecto. Pierdes 5 PS.");
        System.out.println("La respuesta correcta era: " + generadorDePreguntas.getRespuesta(turno));
        atacante.recibirDano( valor: 5);
        respuestasIncorrectas[atacanteIndex]++;
        if (!atacante.estaVivo()) {
            System.out.println("¡El Sabio " + atacante.getNombre() + " se ha quedado sin PS y fue eliminado!");
        }
    }
  }
}
```

Preguntas:

Preguntas.java × RegistroDeJugadores.java ResumenPartida.java

```java
package MisClases;
import InterfaceAbstracta.Personaje;

public class Preguntas {  4 usages
    private String[] preguntas;  2 usages
    private String[] respuestas;  2 usages
    private String[][] opciones;  3 usages

    public Preguntas() {  2 usages
        preguntas = new String[]{
                "¿Cuanto es 7 + 6?",
                "¿Cuanto es la raiz cuadrada de 81?",
                "¿Cual es la capital de Peru?",
                "¿Cuanto es 5 x 12?",
                "¿Con que palabra se define una clase en Java?",
                "¿Que instrumento mide la temperatura?",
                "¿Quien fue el libertador del Peru?",
                "¿Que funcion se usa para imprimir en consola?",
                "¿Quien escribio 'Romeo y Julieta'?",
                "¿Cuanto es 2 al cubo?",
                "¿Que estudia la cinematica?",
                "¿Cuanto es 100 dividido entre 4?",
                "¿Quien fue el ultimo inca del Tahuantinsuyo?",
                "¿Que estructura se usa para repetir código mientras una condicion sea verdadera?",
                "¿Que evento inicio la Edad Moderna?",
                "¿Cual fue la capital del imperio incaico?",
                "¿Que civilización construyo Machu Picchu?",
                "¿Quien formulo las leyes del movimiento?",
                "¿Que palabra se usa para comparar cadenas de texto?",
                "¿Que velocidad tiene la luz en el vacio (en km/s)?",
                "¿Que imperio construyo el Coliseo Romano?",
                "¿Cual es la formula de la velocidad?",
                "¿Con que palabra se define un arreglo en Java?",
                "¿Que unidad se usa para medir fuerza en el SI?",
                "¿En que año se proclamo la independencia del Peru?",
                "¿En que batalla se consolido la independencia del Peru?",
                "¿Quien lidero el movimiento de independencia de la India?",
                "¿Que año comenzo la Segunda Guerra Mundial?",
                "¿Que año cayó el Imperio Romano de Occidente?",
                "¿Quien fue el emperador frances derrotado en Waterloo?"
        };
        opciones = new String[][]{
                {"a) 11", "b) 12", "c) 13", "d) 14"},
                {"a) 7", "b) 8", "c) 9", "d) 10"},
                {"a) Arequipa", "b) Trujillo", "c) Lima", "d) Cusco"},
                {"a) 50", "b) 60", "c) 55", "d) 65"},
                {"a) define", "b) class", "c) public", "d) static"},
                {"a) Barometro", "b) Termometro", "c) Higrometro", "d) Cronometro"},
                {"a) Bolivar", "b) San Martin", "c) Grau", "d) Bolognesi"},
                {"a) print()", "b) echo", "c) System.out.println", "d) write()"},
```

```java
                {"a) Cervantes", "b) Lope de Vega", "c) Shakespeare", "d) Moliere"},
                {"a) 6", "b) 7", "c) 8", "d) 9"},
                {"a) Movimiento", "b) Genética", "c) Electricidad", "d) Optica"},
                {"a) 20", "b) 25", "c) 30", "d) 35"},
                {"a) Manco Capac", "b) Pachacutec", "c) Atahualpa", "d) Tupac Amaru"},
                {"a) if", "b) switch", "c) for", "d) while"},
                {"a) Guerra Civil", "b) Caida de Roma", "c) Descubrimiento de America", "d) Revolución Francesa"},
                {"a) Lima", "b) Ayacucho", "c) Cusco", "d) Arequipa"},
                {"a) Azteca", "b) Inca", "c) Maya", "d) Nazca"},
                {"a) Newton", "b) Einstein", "c) Galileo", "d) Kepler"},
                {"a) compareTo", "b) ==", "c) equals", "d) isEqual"},
                {"a) 150000", "b) 200000", "c) 250000", "d) 300000"},
                {"a) Griego", "b) Romano", "c) Egipcio", "d) Persa"},
                {"a) v=s/t", "b) v=d*t", "c) v=d/t", "d) v=s*d"},
                {"a) int", "b) list", "c) array", "d) new"},
                {"a) Pascal", "b) Newton", "c) Joule", "d) Watt"},
                {"a) 1821", "b) 1824", "c) 1810", "d) 1820"},
                {"a) Junin", "b) Ayacucho", "c) Tarapaca", "d) Pichincha"},
                {"a) Mandela", "b) Gandhi", "c) Luther King", "d) Lenin"},
                {"a) 1938", "b) 1940", "c) 1939", "d) 1941"},
                {"a) 476", "b) 500", "c) 450", "d) 400"},
                {"a) Alejandro Magno", "b) Julio Cesar", "c) Napoleon", "d) Hitler"},
        };
        respuestas = new String[]{
                "c", "c", "c", "b", "b", "b", "b", "c", "c", "c", "a", "b", "c", "d", "c",
                "c", "b", "a", "c", "d", "b", "c", "d", "b", "a", "b", "b", "c", "a", "c"
        };
    }

    public String getPregunta(int nroPregunta) {  1 usage
        return preguntas[nroPregunta];
    }

    public String getRespuesta(int nroPregunta) {  4 usages
        return respuestas[nroPregunta];
    }

    public void mostrarPreguntaCompleta(int nroPregunta, Personaje jugador, int turno) {  2 usages
        System.out.println("┌──────────────────────────────────────────┬──────────┬──────────┐");
        System.out.printf("│ Jugador: %-24s│ PS:%-4d│ XP:%-3d│\n", jugador.getNombre(), jugador.getPs(), jugador.getXp());
        System.out.println("├──────────────────────────────────────────┴──────────┴──────────┤");
        System.out.printf("│ Pregunta N°%-39s│\n",(turno + 1));
        System.out.println("├──────────────────────────────────────────────────────────────┤");
        System.out.println(" " + getPregunta(nroPregunta));
        for (int i = 0; i < opciones[nroPregunta].length; i++) {
            System.out.println(" " + opciones[nroPregunta][i]);
        }
        System.out.println("└──────────────────────────────────────────────────────────────┘");
    }
}
```

RegistroDeJugadores:

```java
RegistroDeJugadores.java ×   ResumenPartida.java

package MisClases;
import LaAventuraDelSabio.*;
import InterfaceAbstracta.Sabio;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Scanner;

public class RegistroDeJugadores {  4 usages
    private static final Map<Integer, String> ELEMENTOS_MAP = new HashMap<>();  4 usages
    static {
        ELEMENTOS_MAP.put(1, "fuego");
        ELEMENTOS_MAP.put(2, "agua");
        ELEMENTOS_MAP.put(3, "tierra");
    }
```

```java

    public static List<Sabio> registrarJugador(int numJugadores) {  2 usages
        List<Sabio> jugadores = new ArrayList<>();
        Scanner teclado = new Scanner(System.in);
        String nombre;
        int opcionRol;
        for (int i = 0; i < numJugadores; i++) {
            MensajeDurantePartida.mostrarRegistroJugador(i);
            System.out.print("Ingrese su nombre: ");
            nombre = teclado.next();
            String rolElegido;
            do {
                MensajeDurantePartida.mostrarElementosSabio(nombre);
                System.out.print("Opción → ");
                opcionRol = teclado.nextInt();

                rolElegido = ELEMENTOS_MAP.get(opcionRol);

                if (rolElegido == null) {
                    System.out.println("Opción inválida. Intenta de nuevo.");
                }
            } while (rolElegido == null);
            Sabio nuevoSabio = switch (rolElegido) {
                case "agua" -> new SabioAgua(nombre);
                case "tierra" -> new SabioTierra(nombre);
                default -> new SabioFuego(nombre);
            };
            jugadores.add(nuevoSabio);
        }
        return jugadores;
    }

    public static void mostrarInfoJugador(List<Sabio> jugadores) {  2 usages
        System.out.println("\n╔═══════════════════════════════════╗");
        System.out.println("│        DETALLE DE JUGADORES        │");
        System.out.println("╟──────────────┬──────────┬──────┬─────╢");
        System.out.printf("│ %-10s │ %-7s │ %-4s │ %-3s │\n","NOMBRE", "ROL", "PS", "XP");
        System.out.println("╟──────────────┼──────────┼──────┼─────╢");
        for (Sabio sabio : jugadores) {
            System.out.printf("│ %-10s │ %-7s │ %-4d │ %-3d │\n",
                    sabio.getNombre().toUpperCase(),
                    sabio.getRol().toUpperCase(),
                    sabio.getPs(),
                    sabio.getXp());
        }
        System.out.println("╚══════════════╧══════════╧══════╧═════╝");
    }
}
```

ResumenPartida:

```java
ResumenPartida.java ×
package MisClases;
import java.util.List;
import InterfaceAbstracta.Sabio;

public class ResumenPartida {  2 usages
    public static void mostrarResumenSolitario(Sabio jugador, int respuestasCorrectas, int respuestasIncorrectas) {  1 usage
        System.out.println("\n═══════════════════════════════════════════");
        System.out.println("\t\t\tRESUMEN DE TU PARTIDA");
        System.out.println("═══════════════════════════════════════════");
        System.out.printf("Jugador: %s\n", jugador.getNombre());
        System.out.printf("Rol: %s\n", jugador.getRol().toUpperCase());
        System.out.println("-------------------------------------------");
```

```java
        System.out.printf("%-25s %5d\n", "Respuestas Correctas:", respuestasCorrectas);
        System.out.printf("%-25s %5d\n", "Respuestas Incorrectas:", respuestasIncorrectas);
        System.out.printf("%-25s %5d\n", "XP Acumulado:", jugador.getXp());
        System.out.printf("%-25s %5d\n", "PS Final:", jugador.getPs());
        System.out.println("----------------------------------------------");
        if (jugador.getXp() >= 40) {
            System.out.println("\n═══════════════════════════════════════════════");
            System.out.println("Felicidades, has demostrado ser un gran Sabio del " + jugador.getRol());
        } else {
            System.out.println("Aún te falta mucho por mejorar.");
        }
        System.out.println("═══════════════════════════════════════════════");
    }

    public static void mostrarResumenVersus(List<Sabio> jugadores, int[] correctas, int[] incorrectas) { 1 usage
        System.out.println("\n╔═══════════════════════════════════════════════════════════════════════╗");
        System.out.println("║                          RESUMEN FINAL DE PARTIDA                     ║");
        System.out.println("╟═══════╤═══════╤═══════╤═══════╤═══════╤═══════╢");
        System.out.printf("║ JUGADOR ║ ROL   ║ RESP. CORRECTAS ║ RESP. INCORRECTAS ║ XP    ║ PS FINAL ║");
        System.out.printf("╟═══════╤═══════╤═══════╤═══════╤═══════╤═══════╢");
        int maxCorrectas = -1;
        String ganador = "Nadie (todos K.O.)";
        for (int i = 0; i < jugadores.size(); i++) {
            Sabio sabio = jugadores.get(i);
            System.out.printf("║ %-10s ║ %-10s ║ %-15d ║ %-18d ║ %-6d ║ %-8d ║\n",
                    sabio.getNombre(),
                    sabio.getRol().toUpperCase(),
                    correctas[i],
                    incorrectas[i],
                    sabio.getXp(),
                    sabio.getPs());
            if (correctas[i] > maxCorrectas) {
                maxCorrectas = correctas[i];
                ganador = sabio.getNombre();
            }
        }
        System.out.printf("╟═══════╤═══════╤═══════╤═══════╤═══════╤═══════╢");
        long jugadoresVivos = jugadores.stream().filter(Sabio::estaVivo).count();
        if (jugadoresVivos == 1) {
            ganador = jugadores.stream().filter(Sabio::estaVivo).findFirst().get().getNombre() + " (Único Sobreviviente)";
        } else if (maxCorrectas > -1) {
            ganador = "El más Sabio: " + ganador;
        }
        System.out.printf("\nResultado: %s con %d respuestas correctas.\n", ganador, maxCorrectas);
    }
}
```

4. EJECUCION DEL CODIGO:

```
Run    Main ×

C:\Users\M&M\.jdks\openjdk-24.0.1\bin\java.exe "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA Community Edition 2025.1.2\lib\idea_rt.jar=50

╔═══════════════════════════════╗
║         LA AVENTURA DEL SABIO         ║
╟═══════════════════════════════╢
║ 1. Solitario (contra la máquina)      ║
║ 2. Versus (2 o 3 jugadores)           ║
║ 9. Salir del Juego                    ║
╚═══════════════════════════════╝

Elige una opción: 2
---------Modo Versus seleccionado---------
Ingrese cantidad de jugadores (2 o 3): 2


╔═══════════════╗
║    REGISTRO DE JUGADOR 1    ║
╚═══════════════╝

Ingrese su nombre: BENITES
Hola SABIO BENITES ¿A que elemento quieres pertenecer?:

╔═══╤═══════╗
║ [1] ║ FUEGO   ║
║ [2] ║ AGUA    ║
║ [3] ║ TIERRA  ║
╚═══╧═══════╝

Opción → 1
```

```
╔═══════════════════╗
║   REGISTRO DE JUGADOR 2   ╒
╚═══════════════════╝

Ingrese su nombre: CARLOS
Hola SABIO CARLOS ¿A que elemento quieres pertenecer?:

┌──────┬──────────┐
│ [1]  │ FUEGO    │
│ [2]  │ AGUA     │
│ [3]  │ TIERRA   │
└──────┴──────────┘

Opción → 2


╔═══════════════════════════╗
║      DETALLE DE JUGADORES      ╒
╟───────────────────────────╢
╟──────────┬──────┬────┬────╢
╟ NOMBRE   │ ROL  │ PS │ XP ╢
╟──────────┼──────┼────┼────╢
╟ BENITES  │ FUEGO│ 100│ 0  ╢
╟ CARLOS   │ AGUA │ 70 │ 0  ╢
╚══════════╧══════╧════╧════╝


═══════════════════════════════
    LA AVENTURA COMIENZA... ¡PREPÁRATE!
═══════════════════════════════


┌──────────────────────────┬──────┬──────┐
│ Jugador: BENITES         │ PS:100 │ XP:0 │
├──────────────────────────┴──────┴──────┤
│ Pregunta N°1                            │
├─────────────────────────────────────────┤
 ¿Cuanto es 7 + 6?
 a) 11
 b) 12
 c) 13
 d) 14
└─────────────────────────────────────────┘

 Elige tu respuesta: c
¡Correcto! Ganas 10 XP.
🔥 ¡BENITES (Fuego) usó Erupción! Causó 15 de daño a CARLOS.

┌──────────────────────────┬──────┬──────┐
│ Jugador: CARLOS          │ PS:55  │ XP:0 │
├──────────────────────────┴──────┴──────┤
│ Pregunta N°1                            │
├─────────────────────────────────────────┤
 ¿Cuanto es la raiz cuadrada de 81?
 a) 7
 b) 8
 c) 9
 d) 10
└─────────────────────────────────────────┘

 Elige tu respuesta: a
Incorrecto. Pierdes 5 PS.
La respuesta correcta era: c

┌──────────────────────────┬──────┬──────┐
│ Jugador: BENITES         │ PS:100 │ XP:10 │
├──────────────────────────┴──────┴──────┤
│ Pregunta N°2                            │
├─────────────────────────────────────────┤
 ¿Cual es la capital de Peru?
 a) Arequipa
 b) Trujillo
 c) Lima
 d) Cusco
└─────────────────────────────────────────┘

 Elige tu respuesta: b
Incorrecto. Pierdes 5 PS.
La respuesta correcta era: c

┌──────────────────────────┬──────┬──────┐
│ Jugador: CARLOS          │ PS:50  │ XP:0 │
├──────────────────────────┴──────┴──────┤
│ Pregunta N°2                            │
├─────────────────────────────────────────┤
 ¿Cuanto es 5 x 12?
 a) 50
 b) 60
 c) 55
 d) 65
└─────────────────────────────────────────┘

 Elige tu respuesta: c
Incorrecto. Pierdes 5 PS.
La respuesta correcta era: b
```

```
┌──────────────────────────────────────┬──────────┬──────────┐
│ Jugador: BENITES                     │ PS:77    │ XP:20    │
├──────────────────────────────────────┴──────────┴──────────┤
│ Pregunta N°6                                                │
├─────────────────────────────────────────────────────────────┤
 ¿Que estudia la cinematica?
 a) Movimiento
 b) Genética
 c) Electricidad
 d) Optica
└─────────────────────────────────────────────────────────────┘

 Elige tu respuesta: a
¡Correcto! Ganas 10 XP.
🔥 ¡BENITES (Fuego) usó Erupción! Causó 15 de daño a CARLOS.

┌──────────────────────────────────────┬──────────┬──────────┐
│ Jugador: CARLOS                      │ PS:5     │ XP:10    │
├──────────────────────────────────────┴──────────┴──────────┤
│ Pregunta N°6                                                │
├─────────────────────────────────────────────────────────────┤
 ¿Cuanto es 100 dividido entre 4?
 a) 20
 b) 25
 c) 30
 d) 35
└─────────────────────────────────────────────────────────────┘

 Elige tu respuesta: c
Incorrecto. Pierdes 5 PS.
La respuesta correcta era: b
¡El Sabio CARLOS se ha quedado sin PS y fue eliminado!
La partida ha terminado anticipadamente, solo queda un Sabio.

╔═══════════════════════════════════════════════════════════════════════════════╗
║                              RESUMEN FINAL DE PARTIDA                         ║
╔═══╦═══╦═══╦═══╦═══╦═══╗
║ JUGADOR ║ ROL     ║ RESP. CORRECTAS ║ RESP. INCORRECTAS ║ XP    ║ PS FINAL ║
╔═══╦═══╦═══╦═══╦═══╦═══╗
║ BENITES ║ FUEGO   ║ 3               ║ 3                 ║ 30    ║ 77       ║
║ CARLOS  ║ AGUA    ║ 1               ║ 5                 ║ 10    ║ 0        ║
╚═══╩═══╩═══╩═══╩═══╩═══╝

Resultado: BENITES (Único Sobreviviente) con 3 respuestas correctas.

¿Deseas volver a jugar?
[1] Sí
[2] Volver al menú principal
```

5. CONCLUSIONES

- El desarrollo de La Aventura del Sabio permitió implementar con éxito los principales conceptos de la POO como clases, objetos, encapsulamiento, herencia, polimorfismo, interfaces y clases abstractas. Esto se ve reflejado en la estructura modular y reutilizable del código fuente.
- La planificación y ejecución del juego educativo implicaron el diseño de lógica por turnos, validación de entradas, administración de flujo de juego, y estructuración de datos mediante arreglos. Todo ello fortaleció el pensamiento lógico del equipo.
- Se logró combinar un enfoque lúdico con fines educativos, utilizando preguntas clasificadas por dificultad, otorgando experiencia, niveles y feedback en tiempo real, lo que promueve el aprendizaje significativo.
- A pesar de no incluir una interfaz gráfica, se desarrolló un sistema funcional, claro, sin errores críticos, y con un flujo de interacción amigable para 1 a 3 jugadores.
- El proyecto fomentó el trabajo en equipo, la distribución de tareas, revisión cruzada de código, y mejora continua del diseño del juego durante el desarrollo.

6. RECOMENDACIONES

- Antes de programar, piensa: ¿Qué entidad quiero representar? ¿Un jugador, una pregunta, un carro, un cliente?
- Dale atributos (variables como nombre, edad, xp) y acciones (métodos como responderPregunta(), subirNivel()).
- Esto te ayudará a entender mejor cómo se estructura la lógica en POO.